KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using a Transformer Encoder Architecture

Markus Swegmark

Supervisor: TBD
Examiner: TBD
Host: TBD

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

This thesis investigates joint clustering of radar emitters and their operating modes from intercepted pulse descriptor word (PDW) streams using a transformer encoder architecture.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, supervised contrastive learning.

Sammanfattning

Detta examensarbete undersöker gemensam klustring av radarsändare och deras operativa moder från avlyssnade pulsbeskrivande ord (PDW) med hjälp av en transformerbaserad encoderarkitektur.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, självövervakad inlärning.

Acknowledgments

I would like to thank my supervisor and examiner at KTH, and the host organization for their support throughout this thesis project.

Stockholm, May 2026
Markus Swegmark

Contents

Abstract i

Sammanfattning ii

Acknowledgments iii

List of Acronyms ix

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Research Questions . . . 2
- 1.4 Objectives and Scope . . . 2
  - 1.4.1 Objectives . . . 2
  - 1.4.2 Delimitations . . . 3
- 1.5 Contributions . . . 3
- 1.6 Ethical and Societal Considerations . . . 3
- 1.7 Thesis Outline . . . 4

2 Background 5
- 2.1 Radar Systems and Passive Reception . . . 5
  - 2.1.1 Electromagnetic Pulses for Navigation, Surveillance, and Control 5
  - 2.1.2 Time-of-Flight Ranging and Ambiguity Fundamentals . . . 5
  - 2.1.3 Transmitting radars versus receive-only systems . . . 6
  - 2.1.4 From the raw signal to a pulse sequence and descriptor words . 7
  - 2.1.5 Physical sources and transmit schedules . . . 8
- 2.2 Signal Processing Pipeline for Passive ELINT . . . 9
  - 2.2.1 Front-end reception, detection, and pulse-parameter estimation . 9
  - 2.2.2 Deinterleaving and pulse-train formation . . . 9
  - 2.2.3 Emitter reporting and equipment identification . . . 10
  - 2.2.4 Operating-mode analysis and waveform libraries . . . 10
  - 2.2.5 Emitter geolocation, fusion, and downstream displays . . . 10
- 2.3 Machine Learning Fundamentals . . . 11
  - 2.3.1 Learning paradigms . . . 11
  - 2.3.2 Representation learning . . . 12
  - 2.3.3 Applications in radar deinterleaving and mode classification . . 12
- 2.4 Clustering Methods . . . 13
- 2.5 Deep Representation Learning . . . 13

2.6 The Transformer Encoder 14

3 Related Work **16**
3.1 Classical Radar Emitter Identification 16
3.2 Deep Learning for Radar Signal Analysis 16
3.3 Deep Clustering and Joint Representation Learning 16
3.4 Transformers for Time Series and Sets 17
3.5 Joint and Multi-Task Clustering 17

4 Method **18**
4.1 Problem Formulation 18
4.2 Data Representation and Preprocessing 21
4.2.1 Scenario simulation and pulse records 21
4.2.2 Optional receiver-side degradation 22
4.2.3 Windowing and context length 23
4.2.4 Per-feature scaling 23
4.2.5 Augmentations for contrastive views 24
4.3 Transformer Encoder Architecture 24
4.4 Loss Function 27
4.5 Training Procedure 30
4.6 Evaluation Metrics 31
4.7 Baseline Methods 32

5 Experiments and Results **34**
5.1 Experimental Setup 34
5.2 Datasets 34
5.2.1 Qualitative properties of the synthetic corpus 35
5.3 Hyperparameters and Training Details 36
5.4 Quantitative Results 36
5.5 Ablation Studies 37
5.6 Qualitative Analysis 37

6 Discussion **38**
6.1 Interpretation of Results 38
6.2 Comparison with Related Work 38
6.3 Limitations 39
6.4 Implications 40

7 Conclusion **41**
7.1 Summary 41
7.2 Answers to the Research Questions 41
7.3 Future Work 41

References **42**

A Additional Results **45**

B Implementation Details **46**

C Notation Reference **47**

List of Figures

2.1 Illustration of the time relationship in monostatic radar: a transmission (TX) and its received echo are separated by a delay τ corresponding to twice the target range divided by the propagation speed, as in Equation (2.1). 6

2.2 Schematic of the parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n, and a carrier RF f_n, sketched as the fast oscillation inside the middle pulse; successive pulses also define a PRI (equivalently a PRF $= 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\boldsymbol{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ that is the usual input to downstream emitter and mode reasoning [1]. 8

4.1 Illustration of a PDW stream $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ for one pulse train, in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text for the train-local emitter labels; the mode legend uses grayscale pattern swatches with black text for the global mode catalogue. Emitter recovery seeks the within-train partition into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode prediction labels each pulse with its global mode m_n, while allowing mode hops within a fixed emitter. 19

4.2 Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\text{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \to 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$. 25

List of Tables

5.1 Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest. 35
5.2 Primary optimisation hyperparameters for the proposed model. 36

List of Acronyms

ACC clustering accuracy

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

EW electronic warfare

FFN feed-forward network

HDBSCAN Hierarchical density-based spatial clustering of applications with noise

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

NMI normalized mutual information

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RADAR radio detection and ranging

RF radio frequency

TOA time of arrival

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Recent large-scale conflicts illustrate how contested the electromagnetic spectrum has become. Commanders rely on timely, passive sensing to build situational awareness, protect platforms, and coordinate fires without revealing their own emissions. Within EW, which aims to exploit, protect, and maneuver in the spectrum, ESM systems listen passively and supply the measurements that feed higher-level ELINT tasks such as detecting radars, grouping pulses by emitter, and inferring behavior [1, 2].

Modern radars are increasingly agile: they vary carrier frequency, pulse width, amplitude, and PRI across bursts to improve performance and survivability. That agility blurs simple fingerprints and stresses classical rule-based parsers tuned to stable pulse trains. The same physical emitter also switches *modes*—recurrent control policies that map an operational objective (for example, volume search versus track) to a characteristic regime in the pulse train. Mode awareness is therefore operationally informative: it constrains how the waveform may evolve, signals intent at a coarse level, and can help disambiguate emitters when type-level signatures overlap.

At the same time, ML and DL have become the default toolkit for sequence modeling and representation learning in other domains. Radar pulse analysis naturally presents as a stream of low-level measurements—often summarized as PDWs—where long-range dependence and mixture structure (multiple emitters superposed in time) favor learned encoders over purely hand-crafted feature pipelines, even when labels exist only for development or evaluation data.

This thesis studies whether a transformer-style encoder, trained with objectives suited to clustering, can produce embeddings in which both distinct emitters and their operating modes emerge as separable structure. The remainder of the introduction states the problem precisely, lists the research questions, and outlines scope, contributions, and ethics.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from multiple emitters are interleaved and each emitter may change operating mode over time (Section 1.1). Operators ultimately require both emitter grouping and operating-mode information from the same measurement stream, and classical pipelines therefore chain deinterleaving with separate mode heuristics or specialised supervised classifiers; under

agile waveforms and overlapping trains, that separation scales poorly and does not guarantee a single representation whose geometry exposes emitter- and mode-level structure simultaneously. This thesis addresses the joint problem in a supervised setting: training pulse trains are generated by a controlled scenario simulator (Section 4.2) in which every PDW carries two ground-truth labels with asymmetric scope — a per-pulse emitter identity that is unique only *within* the pulse train that contains the pulse, and an operating mode drawn from a finite catalogue that is shared across all pulse trains. The task is therefore to learn from this label structure a single representation in which (i) the train-local emitter labels can be used to cluster pulses by physical emitter at inference time, including on previously unseen trains whose number of emitters is not known in advance, and (ii) the global mode catalogue can be used to predict the operating mode of each pulse, while remaining robust to mixture effects, noise, and imperfect preprocessing.

1.3 Research Questions

The thesis addresses the following research questions:

RQ1 Under practical sequence-length limits of a transformer encoder, can the model support effective deinterleaving and recover operating modes via clustering?

RQ2 Does a joint objective that couples deinterleaving with mode-aware clustering yield better clustering performance or representations than a comparable single-objective transformer baseline focused on deinterleaving alone?

1.4 Objectives and Scope

1.4.1 Objectives

The work pursues objectives that operationalize the research questions in Section 1.3. First, we specify and implement a transformer-style encoder that ingests bounded windows of PDWs from interleaved pulse streams and maps them to a dual embedding (Section 4.1), together with a post-hoc clustering stage for the emitter branch and a classification head over the global mode catalogue for the mode branch. We then study how performance depends on practical limits on context length, mixture density, and noise, reporting agreement with the ground-truth simulator partitions using standard scores such as NMI and ARI (Chapter 5). Second, we formulate at least one joint training objective that couples a within-train supervised contrastive term for emitter identity with a global classification term for operating mode, and compare it against a matched baseline trained primarily for deinterleaving or representation quality without that coupling, holding architecture capacity and data conditions as constant as feasible (Chapters 4 and 5).

1.4.2 Delimitations

The scope is deliberately narrow. We operate at the level of PDWs or equivalent tabular pulse features supplied to the model; pulse detection, RF conditioning, antenna-level processing, and geometry-driven fusion across platforms are out of scope. The focus is offline or batch analysis of recorded streams rather than hard real-time implementation on embedded hardware, and we do not address full operational emitter naming against classified libraries. That choice excludes an *online* regime in which models update continuously from a live pulse stream under latency and memory budgets: training and evaluation here assume access to bounded windows that can be revisited, shuffled, and mini-batched, which sidesteps causal filtering, drift handling, and resource accounting that a streaming deployment would require. Consequently, conclusions concern representation quality and clustering agreement on curated windows rather than end-to-end tracking performance over arbitrarily long horizons. Training relies on the simulator-provided emitter and mode labels (Section 4.2); at inference time the trained encoder is intended to operate on previously unseen pulse trains whose emitter membership is recovered by clustering and whose operating modes are read from the classifier head described in Section 1.2. Finally, we do not claim field validation on classified measurements; conclusions are drawn from synthetic or curated datasets together with stress tests that mimic deinterleaving noise (Chapter 5).

1.5 Contributions

The main contributions of this thesis are:

- Placeholder contribution.
- Placeholder contribution.

1.6 Ethical and Societal Considerations

Methods for passive ESM and ELINT sit squarely in dual-use territory: the same representations that support defensive situational awareness, spectrum management, and safety-of-navigation applications can, in principle, support targeting or surveillance against specific platforms if deployed without appropriate policy, oversight, and legal constraints. This thesis develops ML techniques on abstract pulse-level features and does not address deployment, rules of engagement, or classification of live operational data; nevertheless, the capability to group emitters and recognise operating modes from PDW streams should be understood as a general-purpose building block whose societal consequences depend on the institutional context in which it is used.

From a data-ethics perspective, the experimental work relies on controlled or synthetic PDW streams rather than collections derived from live emitters, which limits

privacy and classification sensitivity concerns at the cost of weaker guarantees about behavior under classified or vendor-specific waveforms. When such methods are extended toward operational data, stewards must enforce access control, purpose limitation, and retention policies consistent with national regulation and organizational policy, and they should document provenance and limitations so that operators do not over-trust cluster labels that lack ground truth.

Training modern DL encoders incurs non-negligible energy use for computation and cooling; relative to lightweight classical deinterleaving heuristics, transformer-scale models increase the carbon footprint of experimentation and of any future always-on training loops. Mitigation follows standard practice: reuse checkpoints, avoid redundant hyperparameter sweeps, prefer smaller models when they meet accuracy targets, and report approximate training cost where it informs reproducibility.

At a societal level, automation of emitter and mode analysis may concentrate analytic capacity in well-resourced actors, widen asymmetries in electromagnetic situational awareness, and reduce the visibility of human expert judgment if clusters are treated as authoritative without audit trails. Responsible research therefore favors transparent evaluation protocols, clear statements of failure modes under noise and mixture, and conservative claims about operational readiness.

1.7 Thesis Outline

The remainder of the thesis is organized as follows. Chapter 2 supplies textbook-style background on pulsed radar, PDWs, emitters and operating modes, the ELINT processing chain, and the ML and transformer concepts needed to read the method. Chapter 3 turns to the literature, contrasting classical emitter identification with deep supervised radar models, deep clustering objectives, sequence transformers, and work on joint or hierarchical cluster structure; each subsection closes with the limitation that motivates the present approach. Chapter 4 states the formal problem, documents data representation and preprocessing (Section 4.2), specifies the dual-branch encoder and losses (Sections 4.3 and 4.4), describes training and metrics, and lists baselines. Chapter 5 reports hardware and software setup, the synthetic scenario corpus (Section 5.2), hyperparameters, quantitative scores, ablations, and qualitative analyses. Chapter 6 interprets findings, positions them against prior art, states limitations—including simulator-only evidence and operational gaps—and discusses implications. Chapter 7 summarizes answers to the research questions and outlines future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

2.1.1 Electromagnetic Pulses for Navigation, Surveillance, and Control

The term RADAR stands for **Ra**dio **D**etection **a**nd **R**anging. At its core, a radar system operates by sending out electromagnetic pulses and then listening for the echoes that reflect back from objects in the environment. This basic principle supports a wide range of applications, including air traffic control, weather monitoring, altitude measurement, navigation of ships and vehicles, target tracking, and monitoring of the electromagnetic spectrum alongside commercial communication services [2].

What unifies all radar systems is a practical goal: they emit a carefully controlled burst of electromagnetic energy, allow it to scatter from surrounding objects, and then extract information about the location and motion of those objects from the returning signals. Electromagnetic waves are generated whenever electric charges accelerate, such as when an alternating current passes through an antenna. These waves move outward from the antenna at the speed of light in air. In radar, each outgoing pulse can be thought of as a brief and well-defined packet of energy. By recording the time it takes for this energy to bounce off an object and return, and knowing the speed at which it traveled, the radar can directly calculate how far away the object is [2].

2.1.2 Time-of-Flight Ranging and Ambiguity Fundamentals

At its core, radar sensing uses the physics of wave propagation to measure distance. A transmitter emits a brief electromagnetic pulse, which travels at speed c (the speed of light in air), reflects from an object, and returns as an echo. By measuring the elapsed time τ between emission and reception—known as the round-trip delay—the system can infer the distance to the object.

In the canonical configuration, a single (monostatic) radar platform handles both transmission and reception, with the wave traversing to the target and back. The fundamental relation connecting time delay to range follows directly from the definition of speed:

$$R = \frac{c\tau}{2}, \tag{2.1}$$

where R is the target range and c is the wave speed [2]. The factor of 2 accounts for the outbound and return paths.

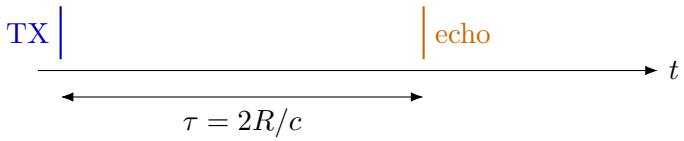

Figure 2.1. Illustration of the time relationship in monostatic radar: a transmission (TX) and its received echo are separated by a delay τ corresponding to twice the target range divided by the propagation speed, as in Equation (2.1).

A similar approach is found in animal echolocation: for instance, a bat produces a short sound pulse and estimates the distance to an obstacle or prey by interpreting the delay before it hears the echo, substituting sound waves for electromagnetic ones [2].

While monostatic setups are common, many practical radar and electronic support systems employ multiple spatially separated receive antennas, or distributed arrays, forming so-called bistatic or multistatic configurations. By correlating the signals arriving at different locations, these systems can measure angles of arrival and other spatial properties in addition to range, thus supporting direction finding and improved localization.

Practical radar systems emit periodic pulses, each separated by a pulse repetition interval (PRI) or, equivalently, a pulse repetition frequency (PRF). A core challenge appears when echoes from earlier pulses arrive after a new pulse is sent. This leads to *range ambiguity*, where it is unclear which transmitted pulse a received echo should be attributed to. The largest range without ambiguity—the *unambiguous range*—is set by the time between pulses:

$$R_{\mathrm{u}} = \frac{c}{2\,\mathrm{PRF}} = \frac{c\,\mathrm{PRI}}{2}. \tag{2.2}$$

Signal processing techniques such as matched filtering and coherent integration increase detection performance within this range, but no algorithm can resolve targets unambiguously beyond R_{u} as limited by the physics of pulse timing [2].

2.1.3 Transmitting radars versus receive-only systems

Any receiver within range can detect intentional electromagnetic transmissions, and repeated radar pulses make timing and direction information available not only to the radar user, but to anyone listening in [1]. This exposes a fundamental trade-off: transmitting enables the measurement of objects at a distance, but simultaneously reveals the radar's presence, position, and activity to others in the environment.

Traditional (transmitting) radar systems measure the delay between their own outgoing pulses and the returning echoes, allowing direct range estimation [2]. In contrast, passive systems such as ESM and ELINT do not transmit their own signals. Instead, they monitor emissions from other sources and attempt to infer information about those emitters and their behavior [1]. Because they do not transmit, passive receivers maintain a lower profile and avoid revealing themselves by radiation—a key

advantage in situations where stealth is important.

Lacking direct control over the transmitted waveform, passive receivers rely on whatever pulse measurements they can extract after detection: carrier frequency, time of arrival, pulse width, amplitude, and angle of arrival if available. This necessity motivates the PDW abstraction described next. Section 2.2 shows how PDWs fit into the overall signal processing chain.

2.1.4 From the raw signal to a pulse sequence and descriptor words

Every radar or receiver starts by collecting a raw electrical signal—essentially a voltage that varies in time as electromagnetic waves are picked up by the antenna. This raw signal can be viewed as a digital recording of the environment's radio activity, capturing everything present, including both the pulses of interest and background noise.

To extract information, the system looks for brief increases in signal energy that stand out above the noise. These increases correspond to pulses—short bursts of transmitted energy. Locating a pulse means finding where in time the signal rises above a set threshold. After locating pulses, the receiver estimates several physical properties for each one: when it arrived (its time of arrival), how long it lasted (its pulse width), how strong it was (amplitude or energy), and what frequency it had. If the receiver has directional antennas, it can also estimate the angle from which the pulse arrived.

After initial detection and measurement, each pulse is summarized into a standardized set of numbers called a *PDW* (pulse descriptor word). This record captures the essential measured properties of a pulse, such as:

$$\boldsymbol{x}_n = \big(f_n,\, \mathrm{PW}_n,\, t_n,\, A_n,\, \theta_n,\, \ldots\big)^\top \in \mathbb{R}^d, \tag{2.3}$$

where f_n is the measured carrier frequency, PW_n is pulse width, t_n is time of arrival (often the leading edge), A_n is amplitude, and θ_n is angle of arrival if available. The ellipsis allows for extra recorded properties, like elevation angle of arrival, but the main idea is that each column represents a specific physical measurement, and the format is fixed once chosen. Figure 2.2 visualizes this five-parameter template.

A sequence of such pulses, indexed by order of arrival $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots)$, forms what is called a *pulse train*. For pulses that come from a single transmitter operating in a stable mode, the intervals between arrivals ($\tau_n = t_n - t_{n-1}$, for $n \geq 2$) are usually regular or follow a set schedule. This interval is called the pulse repetition interval (PRI); its inverse, the pulse repetition frequency (PRF), describes how often pulses are sent:

$$\tau_n = t_n - t_{n-1}, \qquad n \geq 2. \tag{2.4}$$

While simple systems use a fixed interval, more complex transmitters may change their intervals in a predictable or agile way, and this information can be included in the

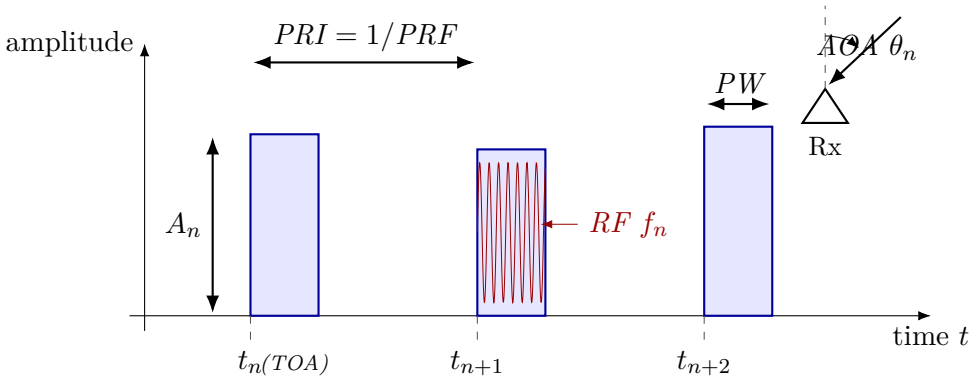

Figure 2.2. Schematic of the parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n, and a carrier RF f_n, sketched as the fast oscillation inside the middle pulse; successive pulses also define a PRI (equivalently a PRF $= 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\boldsymbol{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ that is the usual input to downstream emitter and mode reasoning [1].

record for each pulse.

Each measured descriptor is thus a function of both the true (but unknown) properties of the transmitting source and the effects of noise, measurement error, and possible missing data. Very weak pulses may be missed, and noise or spurious events may be incorrectly identified as pulses.

Finally, rather than carrying forward the full raw signal, the system represents each pulse only by its measured properties in the form of a time-ordered sequence of PDWs. This sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$—sometimes called the pulse train—is the main input to later processing and learning algorithms. Each $\boldsymbol{x}_n$ is a compact summary of one pulse, and the order reflects the timeline of arrivals, replacing the need for direct inspection of the original sampled voltage trace.

2.1.5 Physical sources and transmit schedules

In passive ESM and ELINT, a *radar emitter* means a physical transmitter whose pulses line up in time and frequency strongly enough to be treated as one source [1]. An *operating mode* is the instantaneous transmit recipe: carrier plan, pulse width, repetition pattern, beam motion, and any programmed switching rules among them [1, 2]. The same hardware cycles among modes as tasks change, so a timeline of PDWs mixes mode segments even when the physical box is fixed. Different platforms can also implement the same functional mode with different numerical parameters, for example two trackers with distinct nominal PRFs [1].

Illustrative mode families include wide-area search, acquisition and track, fire-control illumination, weather sensing, and terminal guidance support [2]. Those roles push different combinations of PRI, dwell, agility, and scan law, which is why operators care

both about identity and about what the transmitter is doing at a given time [1, 2].

The relationship between emitters and modes is many-to-many: one emitter runs many modes over a mission, and different emitters can realize similar modes with different parameter sets [1]. The rest of this chapter walks through the passive processing chain (Section 2.2) and then supplies the learning background used in the method.

2.2 Signal Processing Pipeline for Passive ELINT

In EW and ESM applications, measured radar energy passes through a staged *signal processing pipeline* before it reaches higher-level ELINT reasoning. The exact implementation varies with platform, band, and mission software, but the logical chain is broadly stable across textbooks and system descriptions.

2.2.1 Front-end reception, detection, and pulse-parameter estimation

At the *front end*, antennas and RF conditioning capture emissions over one or more instantaneous bandwidths, often after wideband digitization or channelized sub-band processing. A *detection* stage then marks pulse-like events against a noise and clutter background, producing a sparse list of candidate pulse times (and sometimes coarse frequency hints).

Given detections, *parameter estimation* attaches per-pulse measurements such as carrier frequency, pulse width, time of arrival, direction of arrival when available, and amplitude or signal-to-noise proxies, this becomes our PDWs after possibly attatching AoA after triangulation. These estimates are noisy, can be missing for weak emitters, and may include false alarms that are not associated with any physical radar.

2.2.2 Deinterleaving and pulse-train formation

Sorting and *deinterleaving* group pulses into coherent pulse trains by exploiting periodic structure in inter-pulse timing and consistent parameter tracks across time. The output of this stage is commonly packaged as a stream of PDWs (or closely related pulse descriptor records), which form the usual interface to downstream tasks such as emitter identification and mode analysis (Section 2.1.4).

Operationally, deinterleaving developed from deterministic temporal rules toward multi-parameter clustering and, more recently, toward learned sequence models [3]. In comparatively sparse environments with simple pulse trains, PRI inferred from TOA differences often behaved like a stable timing signature, so early algorithms searched for repetition by accumulating histograms of inter-pulse intervals. Cumulative and sequential difference histograms (CDIF and SDIF) are representative: they “vote” for candidate PRI values through peaks in the difference domain and remain standard

pedagogical references [1, 3]. Their accuracy rests on approximately regular timing; dense mixing, harmonic ambiguity, false peaks from cross-emitter differences, and missing pulses that break sequential differencing all erode reliability [3].

Once PRI agility became widespread, many systems augmented timing with unsupervised structure in joint spaces of RF, pulse width, AOA, and related measurements, using methods such as k-means or density-based clustering [3, 4]. That shift treats deinterleaving as geometric separability in hand-crafted feature space, but performance still depends on scaling, metric choices, and hyperparameters (for example cluster counts or neighborhood radii) that are difficult to set when emitters are unknown a priori [3].

Modern emitters compound the difficulty through overlapping pulse trains, rapid mode changes, and low-observable emissions that increase loss and measurement jitter [3]. The PDW representation in Section 2.1.4 and the learning-based tools in later sections of this chapter target reasoning under those conditions.

In classical ELINT architectures, specifications therefore treat train formation as an early, largely discrete step: deinterleaving is completed first so that downstream logic receives one dominant hypothesis per physical emission path, or a short ranked list, rather than a time-interleaved mixture [1].

2.2.3 Emitter reporting and equipment identification

Emitter reports are assembled after train formation, summarizing each train with aggregated PDW statistics, tentative equipment identities from lookup tables, and temporal extent. Library-driven identification and threat correlation consume those reports together with operator context.

2.2.4 Operating-mode analysis and waveform libraries

Mode classification is then commonly framed as matching the train against a *mode library* of nominal RF bands, PRI/PRF families, stagger laws, and scan programs, so that declared modes remain anchored to catalogued waveform templates. When labels are incomplete, unsupervised or weakly supervised grouping over pulse trains plays the same operational role as *mode clustering* at the library boundary: it proposes coherent behavior segments that analysts later align to doctrine or intelligence products.

2.2.5 Emitter geolocation, fusion, and downstream displays

Positional estimation—for example AOA intersections from direction-finding networks, time-difference-of-arrival fixes along baselines, or fused tracks across collectors—consumes the same deinterleaved associations, which isolates geometry and timing errors from the sorting stage. Later stages may maintain libraries, threat lists, and operator

displays, but for the learning problem in this thesis the critical contract is that the model consumes the deinterleaved PDW stream produced here.

2.3 Machine Learning Fundamentals

At its most general, ML replaces hand-coded decision rules with parametric models whose parameters are fitted to data so that a chosen performance criterion is optimized on observed examples of the task. A model is a function $f_{\boldsymbol{\theta}}\colon \mathcal{X} \to \mathcal{Y}$ from an input space $\mathcal{X}$ to a task-specific output space $\mathcal{Y}$, and learning consists of searching over parameter vectors $\boldsymbol{\theta}$ until $f_{\boldsymbol{\theta}}$ is consistent with the regularities present in the training data. What changes between problems is the nature of the supervision available, the structure of $\mathcal{Y}$, and the loss that scores predictions against that supervision.

A common formal frame is *empirical risk minimization*. Given a dataset $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$ drawn from an unknown distribution and a pointwise loss ℓ, the learner solves

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \; \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i),\, y_i\Big) + \Omega(\boldsymbol{\theta}), \tag{2.5}$$

where Ω is an optional regularizer that trades fidelity to the training data against complexity of the fitted model. Generalization to fresh inputs is then assessed on held-out data, and the entire training procedure—data, model class, loss, optimizer, regularization—is judged by its effect on that out-of-sample behavior rather than on the training fit alone.

2.3.1 Learning paradigms

ML problems are conventionally separated by the kind of supervision available at training time. In *supervised* learning, each $\boldsymbol{x}_i$ is paired with a label y_i, categorical for classification or real-valued for regression, and ℓ penalizes disagreement between the prediction and that label. In *unsupervised* learning, labels are absent and the goal is to expose structure in the input distribution itself, for example through density estimation, dimensionality reduction, or clustering; Section 2.4 treats clustering specifically because it is the central downstream consumer of the embeddings used in this thesis. *Self-supervised* learning sits between the two: it constructs supervisory targets automatically from the data through a designed pretext task—predicting masked parts of an input, contrasting augmented views, or ordering shuffled tokens—so that an encoder can be trained without manual annotation. The remainder of this thesis operates in the supervised regime: training pulse trains carry per-pulse emitter and operating-mode labels (Sections 4.1 and 4.2), and the training objective uses both label streams directly; the unsupervised and self-supervised paradigms are summarised here only because the clustering tools in Section 2.4 and the contrastive representation-learning machinery in Section 2.5 are inherited from those traditions.

2.3.2 Representation learning

Much of modern DL practice is best understood as *representation learning*: a parametric encoder maps high-dimensional observations into a lower-dimensional vector space whose geometry is shaped by the training objective. Downstream tasks—classification, clustering, retrieval, or sequence prediction—then operate on the resulting *embeddings* $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) \in \mathbb{R}^d$ rather than on raw sensor coordinates. The advantage of routing tasks through a learned representation is that invariances, similarities, and task-relevant factors of variation can be encoded by $f_{\boldsymbol{\theta}}$ once, after which simple linear or geometric operations on $\boldsymbol{z}$ suffice for many downstream questions. Sections 2.5 and 2.6 expand on this view; the method chapter (Chapter 4) instantiates it for PDW streams, training an encoder whose embeddings are intended to expose emitter and mode structure.

2.3.3 Applications in radar deinterleaving and mode classification

Radar signal deinterleaving and operating-mode classification illustrate the practical pull of these ideas. For decades, both tasks were addressed by deterministic temporal rules and hand-crafted statistics over PDW parameters, as outlined in Section 2.2. As emitters became more agile and electromagnetic environments more crowded, the field migrated first to *shallow* ML in the form of multi-parameter clustering—k-means and density-based methods over RF, AOA, and pulse width—and more recently to *deep* sequence models that learn pulse-level features directly from data [3]. Recurrent and attention-based architectures now treat deinterleaving as sequence labeling in which the per-pulse output identifies an emitter, while transformer encoders in particular are well suited to capturing the non-contiguous, multi-scale relationships induced by jittered, staggered, or grouped PRIs [3, 5].

Mode classification follows a parallel trajectory. Classical ESM systems match deinterleaved pulse trains against catalogued waveform templates in a mode library [1], while learning-based approaches reframe the same task either as supervised classification over known mode labels or as unsupervised grouping over pulse trains when labels are scarce or incomplete [3]. This thesis is positioned in the former, supervised regime for both targets: the operating mode is predicted against a fixed global catalogue per pulse, while the emitter task is trained with explicit per-pulse identity labels whose scope is local to each pulse train, so that the trained embedding can subsequently be clustered within previously unseen trains. The clustering tools in Section 2.4 and the contrastive representation-learning machinery in Sections 2.5 and 2.6 provide the technical foundation that the method chapter builds on.

2.4 Clustering Methods

Placeholder paragraph.

2.5 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map that is trained end-to-end from data. The central object is an *encoder* $f_{\boldsymbol{\theta}}\colon \mathcal{X} \to \mathbb{R}^d$, implemented in practice as a stack of nonlinear layers (for example, convolutions over a grid, recurrence over time, or attention over a sequence) followed by a pooling or readout head that yields a fixed-dimensional embedding $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$. The design question is which training signal makes $\boldsymbol{z}$ useful for the downstream task when labels are scarce or absent.

In supervised learning, $f_{\boldsymbol{\theta}}$ is trained together with a classifier so that $\boldsymbol{z}$ separates classes under a cross-entropy or margin loss. When cluster structure is the target but assignments are unknown, the same encoder can instead be trained with objectives that encourage smoothness, invariance to nuisance transformations, or agreement with a clustering hypothesis in embedding space. *Self-supervised* learning occupies an intermediate regime: the model predicts surrogate targets constructed from the input itself (for example, masked tokens, relative positions, or transformed views), so that $f_{\boldsymbol{\theta}}$ learns semantics without human labels [6].

Contrastive objectives implement instance discrimination by treating two augmentations of the same example as a positive pair and other examples in a minibatch (or a memory bank) as negatives. Let $\boldsymbol{z}_i$ and $\boldsymbol{z}_i^{+}$ denote normalized embeddings of two views of sample i, let $\mathrm{sim}(\cdot,\cdot)$ denote cosine similarity, and let $\tau > 0$ be a temperature. A standard *InfoNCE*-style loss takes the form

$$\mathcal{L}_{\mathrm{NCE}}(i) = -\log \frac{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^{+})/\tau\Big)}{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^{+})/\tau\Big) + \sum_{j \neq i} \exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j^{+})/\tau\Big)}, \tag{2.6}$$

which lower-bounds mutual information between representations of different views under suitable generative assumptions [7]. Large-scale implementations such as SimCLR combine strong augmentations with wide embeddings and many negatives drawn from the same minibatch [6]. Equation (2.6) should be read as a template: the positive set can be enlarged, negatives can be drawn from a queue, and the similarity can be implemented with a learned projection head without changing the underlying contrastive principle.

Pretext tasks offer an alternative route: the network solves an auxiliary problem (predicting rotations, solving jigsaw permutations, inpainting masked regions) whose solution is incidental, but the representation $\boldsymbol{z}$ becomes predictive of semantically stable factors. Such objectives can be combined with clustering when the end goal is partition discovery rather than linear probe accuracy on a fixed label set. Section 3.3 surveys deep

embedded clustering and prototype-based alternatives that couple encoder training with discrete structure, together with the gap relative to nested emitter–mode organization in ELINT streams.

Sequence encoders based on self-attention, including the transformer blocks reviewed in Section 2.6, instantiate $f_{\boldsymbol{\theta}}$ when $\boldsymbol{x}$ is an ordered set of tokens (for example, a window of PDWs). The Method chapter builds on this stack: a contrastive or clustering-friendly loss shapes the embedding space, while the attention-based encoder models temporal context within each window.

2.6 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [8]. Encoder-only stacks, as in BERT [9], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\boldsymbol{Q}$, *keys* $\boldsymbol{K}$, and *values* $\boldsymbol{V}$ to an output matrix of the same leading dimension as $\boldsymbol{Q}$. Each row of $\boldsymbol{Q}$ scores all rows of $\boldsymbol{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\boldsymbol{V}$. With d_k the dimension of each query and key vector (column width of $\boldsymbol{Q}$ and $\boldsymbol{K}$ after the usual transpose convention), the mapping is

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.7}$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [8]. Equation (2.7) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are all linear projections of the same input sequence layout, so a token's representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear map to restore the model width [8]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore add a positional signal to token embeddings before the first layer, for example fixed sinusoidal features or learned position embeddings added element-wise to each token vector [8, 9]. When the raw inputs already carry absolute or relative timing, as is common for irregularly sampled sequences, practitioners sometimes omit a separate positional encoding and rely on those measurements instead; the method chapter returns to this choice for PDW windows.

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [8]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such layers yields a deep encoder that maps an input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$ to contextualised representations $(\boldsymbol{h}_1^{(N)}, \ldots, \boldsymbol{h}_L^{(N)})$, which downstream heads can pool or read out for sequence-level or token-level predictions.

Computational cost. In the usual dense implementation, each self-attention block materialises an $L \times L$ matrix of token–token scores before the softmax, so time and activation memory for that block scale as $\mathcal{O}(L^2)$ in the sequence length L at fixed embedding width, up to constants from the number of heads and projection ranks [8]. Repeating the block across depth multiplies that cost by the number of layers, which motivates hard caps on L, sparse or low-rank attention variants, or chunked processing when inputs are long; the method chapter states the windowing choice adopted here.

CHAPTER 3

Related Work

3.1 Classical Radar Emitter Identification

Placeholder paragraph.

3.2 Deep Learning for Radar Signal Analysis

Placeholder paragraph.

3.3 Deep Clustering and Joint Representation Learning

Classical clustering assumes that either raw features or a fixed embedding already reveal the desired partition. DEC departs from that split by learning an encoder jointly with a soft assignment to a small set of trainable cluster centroids in embedding space [10]. Training alternates between computing soft assignments from the current encoder and refining the encoder with a Kullback–Leibler objective that sharpens confident assignments toward an auxiliary target distribution, so that the network discovers both features and cluster structure. Follow-on work tightens initialization, adds reconstruction constraints, or refines the target distribution; the common theme is that discrete structure is not fixed in advance but co-adapts with the representation.

SwAV and related *prototype* methods replace dense pairwise contrastive negatives with a prediction problem onto a modest codebook of cluster prototypes, swapping predicted codes between augmented views to avoid representation collapse while still encouraging invariance [11]. They sit between purely self-supervised contrastive pretraining [6] and hard assignment k-means: gradients still flow through discrete-like targets, but the geometry is anchored by learned prototypes rather than by instance indices alone.

Several lines extend deep clustering to *multiple* partitions or hierarchical labels, for example by stacking separate assignment heads, alternating optimization over two clusterings, or enforcing orthogonality constraints between subspaces. The gap relevant to this thesis is narrower: emitter identity and operating mode are not arbitrary independent clusterings—modes nest inside emitters—yet many published pipelines still treat a single flat label space or attach two unrelated classification heads to the same backbone. Work that jointly addresses deinterleaving-style separation, in which

emitter identifiers are inherently local to each pulse train and must be recovered by clustering at inference, alongside finer-grained mode prediction against a global mode catalogue, all from a shared transformer encoder, remains comparatively sparse; this combination motivates the supervised dual-branch method in Chapter 4.

3.4 Transformers for Time Series and Sets

Placeholder paragraph.

3.5 Joint and Multi-Task Clustering

Placeholder paragraph.

CHAPTER 4

Method

4.1 Problem Formulation

This chapter studies supervised representation learning from time-ordered streams of PDWs produced by an ESM or deinterleaving front end, as outlined in Section 2.2. Each pulse in the training data carries two ground-truth labels: an *emitter identity*, whose values are unique only within the pulse train that contains the pulse, and an *operating mode*, whose label set is shared across all pulse trains. The goal is twofold: learn a representation from which (i) pulses originating from the same physical emitter within a pulse train can be reliably grouped, and (ii) the global operating mode of each pulse can be recovered.

Observed input

The data is organised as a collection of *pulse trains*, each a finite, time-ordered sequence of PDWs produced during one observation window. A single pulse train of N pulses is represented by feature vectors

$$\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N), \qquad \boldsymbol{x}_n \in \mathbb{R}^d, \tag{4.1}$$

where each $\boldsymbol{x}_n$ encodes the measured pulse parameters associated with one PDW (carrier frequency, amplitude, pulse width, time of arrival, angle of arrival including azimuth and elevation). The model receives one pulse train at a time and produces per-pulse outputs for that train; to keep notation light, we suppress an explicit train index in Equation (4.1) and in what follows.

A single train may contain irregular sampling, missing pulses, and spurious detections inherited from upstream processing; robustness to such effects is a design constraint rather than part of the formal specification.

Per-pulse supervision: emitter and mode labels

Every pulse in the training data is annotated with two ground-truth labels. For each index $n \in \{1, \ldots, N\}$ within a given train, let e_n denote the emitter identity responsible for pulse n and let m_n denote its operating mode. Although both quantities are observed during training, they have fundamentally different semantics.

The mode label $m_n \in \{1, \ldots, M\}$ is drawn from a finite, *global* catalogue of size M that is shared across all pulse trains: the symbol $m_n = \ell$ denotes the same operating

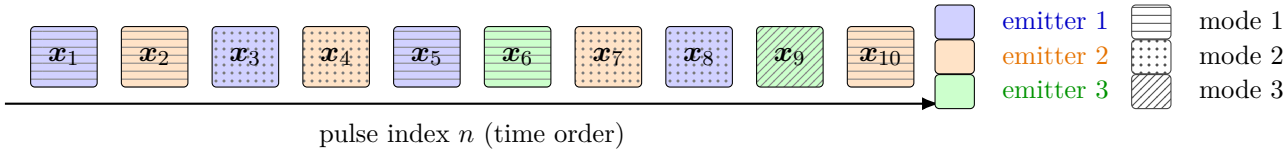

Figure 4.1. Illustration of a PDW stream $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ for one pulse train, in time order. Each cell combines emitter color (fill) and operating-mode pattern; the in-cell vector index is typeset in black. The emitter legend uses color-only swatches with matching colored text for the train-local emitter labels; the mode legend uses grayscale pattern swatches with black text for the global mode catalogue. Emitter recovery seeks the within-train partition into subsets $\mathcal{E}_k$ (constant e_n on each subset); mode prediction labels each pulse with its global mode m_n, while allowing mode hops within a fixed emitter.

mode regardless of which train the pulse belongs to. The emitter label $e_n \in \{1, \ldots, K_{\text{em}}\}$, by contrast, is unique only *within the current train*: the integer indexes the distinct physical emitters encountered in this particular observation window, and the same integer in a different train refers to an unrelated emitter. Consequently, two pulses can be declared to share an emitter only when both belong to the same train, and the supervision provides no notion of a global emitter identity.

The train-specific cardinality K_{em} is observed at training time but varies across trains, and the model is expected to operate on previously unseen trains whose values of K_{em} are unknown a priori. The catalogue size M is fixed and shared across trains, although individual trains typically exercise only a small subset of the M modes.

Induced partitions, PDW streams, and mode hopping

Figure 4.1 sketches a short PDW stream in chronological order for a single pulse train. Fill color encodes the train-local emitter index e_n; pulses that share a color belong to the same emitter even though they need not be contiguous in time (interleaved emissions from several RADAR platforms are typical). Each stream cell uses that emitter fill *and* a grayscale pattern for the global operating-mode label m_n; the vector label $\boldsymbol{x}_i$ inside each cell is drawn in black for legibility. The right-hand legends separate the two cues: emitter entries are color-only (no pattern), and mode entries are monochrome pattern tiles with black text. The formal partitions and mode hopping are stated mathematically in the text below.

Within a pulse train, the emitter supervision induces a partition of $\{1, \ldots, N\}$ into emitter-coherent index sets

$$\mathcal{E}_k = \{\, n \in \{1, \ldots, N\} : e_n = k \,\}, \qquad k \in \{1, \ldots, K_{\text{em}}\}, \tag{4.2}$$

so that $\boldsymbol{x}_n$ and $\boldsymbol{x}_{n'}$ belong to the same emitter-coherent subset if and only if $e_n = e_{n'}$. Each $\mathcal{E}_k$ is the preimage of label k under the map $n \mapsto e_n$, and $\{\mathcal{E}_1, \ldots, \mathcal{E}_{K_{\text{em}}}\}$ is pairwise disjoint with union $\{1, \ldots, N\}$. Because emitter symbols are not shared across trains, this equivalence relation is defined only within each train; the recovery problem at

inference time is to reconstruct it from $\boldsymbol{X}$ alone.

The global mode catalogue induces a second partition,

$$\mathcal{M}_\ell = \{\, n \in \{1, \dots, N\} : m_n = \ell \,\}, \qquad \ell \in \{1, \dots, M\}, \tag{4.3}$$

which groups pulses that share the same operating mode regardless of which physical emitter produced them. The two partitions are coupled because (e_n, m_n) is generated jointly: restricting to a fixed emitter k, the induced set of mode labels

$$\mathcal{L}_k = \{\, m_n : n \in \mathcal{E}_k \,\} \subseteq \{1, \dots, M\} \tag{4.4}$$

has cardinality $L_k = |\mathcal{L}_k|$ bounded by the number of distinct waveform programmes that emitter k exercises. In many RADAR systems, mode hopping switches among a small subset of the catalogue, so L_k is modest—often on the order of single-digit integers—even though the temporal pattern $(m_n)_{n \in \mathcal{E}_k}$ is not constant. The supervised learning problem is then to fit a model whose outputs, on previously unseen trains, recover $\{\mathcal{E}_k\}$ up to an arbitrary relabelling of emitter symbols and assign each pulse to the correct global mode label in $\{1, \dots, M\}$, using the dual embeddings in Equation (4.5).

Desired outputs: dual embeddings and discrete labels

The parametric model considered in this thesis maps an input pulse train to two parallel trajectories in Euclidean embedding spaces,

$$\boldsymbol{Z}^{\mathrm{em}} = (\boldsymbol{z}_1^{\mathrm{em}}, \dots, \boldsymbol{z}_N^{\mathrm{em}}), \quad \boldsymbol{z}_n^{\mathrm{em}} \in \mathbb{R}^p, \qquad \boldsymbol{Z}^{\mathrm{md}} = (\boldsymbol{z}_1^{\mathrm{md}}, \dots, \boldsymbol{z}_N^{\mathrm{md}}), \quad \boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^q, \tag{4.5}$$

with dimensions p and q fixed by the architecture.

The two trajectories play different roles dictated by the label semantics described above. Emitter-oriented vectors $\boldsymbol{z}_n^{\mathrm{em}}$ are trained so that, within each pulse train, embeddings with the same emitter label cluster tightly in $\mathbb{R}^p$ while embeddings with different emitter labels are pushed apart. Because the emitter symbol is train-local, the corresponding supervision is a within-train metric-learning signal rather than a global classifier, and discrete emitter assignments

$$\hat{e}_n \in \{1, \dots, \hat{K}_{\mathrm{em}}\}, \qquad n \in \{1, \dots, N\}, \tag{4.6}$$

are obtained *post hoc* by clustering $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^N$ within each train; the inferred count $\hat{K}_{\mathrm{em}}$ is either selected by the clustering algorithm or supplied from external knowledge. Their main deliverable is a per-pulse emitter index suitable for downstream ELINT workflows such as emitter counting and localisation.

Mode-oriented vectors $\boldsymbol{z}_n^{\mathrm{md}}$, by contrast, are trained against the shared global catalogue of M modes. A classification head $f\colon \mathbb{R}^q \to \mathbb{R}^M$ on top of the embedding

produces the predicted mode

$$\hat{m}_n = \underset{\ell \in \{1,\dots,M\}}{\arg\max}\, f_\ell\left(\boldsymbol{z}_n^{\mathrm{md}}\right), \tag{4.7}$$

and the embedding additionally aims to summarise mode-dependent waveform behaviour so that operating modes can be inspected, compared, or matched against a library of known waveforms.

Architecturally, both streams are produced from a shared encoder $f_{\boldsymbol{\theta}}$ with task-specific heads (see Section 4.3); in deployment, emitter embeddings may be less critical to retain after discrete $\hat{e}_n$ have been fixed, whereas mode embeddings (or intermediate encoder activations feeding the mode head) are natural inputs to specialised mode classifiers.

Learning objective

The optimisation problem combines (i) a supervised within-train metric-learning loss that uses the emitter labels $\{e_n\}$ to shape $\boldsymbol{Z}^{\mathrm{em}}$ and (ii) a supervised classification loss against the global mode catalogue $\{1, \dots, M\}$ that shapes $\boldsymbol{Z}^{\mathrm{md}}$ through the head in Equation (4.7). The two terms exploit the asymmetric label semantics: emitter labels enter the loss only through within-train comparisons, while mode labels are used identically across trains. The concrete loss terms and training protocol are given in Sections 4.4 and 4.5.

4.2 Data Representation and Preprocessing

ELINT data are difficult to obtain at scale, and operational recordings are often sensitive. This work therefore relies on synthetic pulse trains from a proprietary scenario simulator at Saab, which yields controlled scenarios together with full per-pulse supervision of emitter identity and operating mode (Section 4.1); both label streams are consumed directly by the training objective in Section 4.4 as well as by the evaluation metrics in Section 4.6. Earlier KTH degree projects hosted at Saab document synthetic radar-ML pipelines at a comparable level of detail, including scenario-driven pulse generation and explicit noise models [12, 13].

4.2.1 Scenario simulation and pulse records

The simulator produces *scenarios*: time-ordered pulse-level timelines for one or more notional emitters and a receiving platform. Each pulse is described by the continuous fields that enter the PDW representation used here (including TOA, RF, pulse width, and amplitude on a logarithmic scale), together with two discrete labels: an emitter identity that is unique only within the scenario that produced the pulse, and an operating mode drawn from a finite catalogue shared across scenarios (Section 4.1).

Algorithm 1 Optional stochastic degradation of an ordered pulse list (conceptual).

Require: Ordered pulses $\{p_1, \ldots, p_N\}$, drop probability $\rho_{\text{drop}} \in [0, 1)$, spurious probability $\rho_{\text{spu}} \in [0, 1)$
Ensure: Degraded list of pulses preserving time order where applicable
$\mathcal{L} \leftarrow \text{copy}(\{p_1, \ldots, p_N\})$
for each pulse q in $\mathcal{L}$ scanned in forward time order **do**
    if uniform draw $u < \rho_{\text{drop}}$ **then**
        remove q from $\mathcal{L}$
    end if
end for
for each remaining pulse q in $\mathcal{L}$ **do**
    if uniform draw $u' < \rho_{\text{spu}}$ **then**
        insert a spurious pulse after q using the chosen perturbation rule (duplicate with noise or independent draw)
    end if
end for

Jonsson [12] describes the unclassified OpenModesty tool in the same lineage: flight paths and emitter placements define a geometry, pulses are emitted and intercepted one at a time, and the simulator can apply modulations and controlled perturbations consistent with notional radar behavior. The present implementation is proprietary; the important methodological point is that every pulse in the export carries consistent timing and parameter fields so that windows formed below correspond to reproducible segments of a simulated electromagnetic timeline.

Scenarios differ in the number of emitters, dwell in each mode, and simulated horizon so that training covers diverse traffic densities; exact counts and split assignments are deferred to Chapter 5.

4.2.2 Optional receiver-side degradation

Even when emitter physics are ideal in the simulator, downstream ESM processing introduces structured errors. Jonsson [12] distinguishes *dropped pulses*, where a transmitted pulse is not registered by the receiver chain, from *spurious pulses*, where extra pulses appear in a track because deinterleaving or detection stages attach energy to the wrong pulse train. Svensson [13] likewise emphasizes controlled drop, spurious, and additive Gaussian regimes when studying PRI sequences. When the experimental programme applies such degradations, they are implemented as stochastic edits on the ordered pulse list *before* windowing: each pulse may be removed with a scenario- or split-specific drop probability, and spurious insertions may be drawn by copying and perturbing existing pulses or by drawing alternative parameter vectors from a bounded rule set. Discrete grids of drop and spurious rates, rather than only single operating points, simplify reporting across noise sweeps in Chapter 5.

Algorithm 1 is only a bookkeeping template: concrete probabilities, conditioning

on track boundaries, and whether noise is re-drawn per window or fixed per scenario appear in the experiment log together with the simulator export manifest.

4.2.3 Windowing and context length

The encoder operates on fixed-length windows because standard self-attention builds an $L \times L$ compatibility matrix between tokens at each layer, so time and memory scale as $\mathcal{O}(L^2)$ in the window length L at fixed model width, repeated across depth. Let L denote the window length (in pulses) fed to the model; experiments use $L = 1024$. Each scenario is partitioned into contiguous windows of length L. If the remainder after the last full window is shorter than L, it is dropped in the present pipeline; padding with an attention mask would retain the tail without changing the architecture and is left as a straightforward extension.

Training windows are extracted with a stride of $L/2$, so consecutive windows overlap by half their length. This increases the number of training pulses seen per scenario and reduces sensitivity to the particular alignment of window boundaries relative to mode transitions. Validation and test windows use stride L (no overlap), so that metrics are not inflated by near-duplicate windows.

Interleaved traffic and heterogeneous dwell times imply that pulse counts per emitter and per operating mode vary widely inside a window, and the effective PRI structure can make the number of pulses per emitter differ across the corpus even when scenario durations match [12]. Some windows therefore contain few or no pulses from a rare emitter or a short-lived mode, even when that label is frequent in the parent scenario, and dropped tails can remove trailing segments entirely. These effects induce a form of *label incidence imbalance* at the window level that is distinct from corpus-level class histograms and should be read alongside the reporting conventions in Chapter 5.

4.2.4 Per-feature scaling

Amplitude is already provided on a logarithmic scale (decibels). Continuous fields are transformed before windowing so that statistics are computed on the training split only and then applied to validation and test data. The log-amplitude, carrier frequency, and pulse width are standardised to zero mean and unit variance using the training-set mean and standard deviation of each field, so that channels measured in different physical units contribute comparably to dot products in the encoder [12]. Each TOA is first min–max scaled to $[0, 1]$ using training-set bounds; within every window, all rescaled TOA values are shifted by subtracting the TOA of the first pulse so that the window starts at time zero. This removes absolute clock offset while preserving intra-window timing structure.

4.2.5 Augmentations for contrastive views

Contrastive objectives in Section 4.4 consume two stochastic views of each training window. Those views combine the deterministic normalisation above with additional pulse-level or window-level perturbations (for example timing jitter, amplitude scaling, small frequency offsets, or pulse dropout) so that invariances reflect both simulator noise and plausible analogue front-end variability. The exact augmentation set and sampling ranges are part of the training configuration and are listed with other hyperparameters in Section 5.3.

Windows are grouped into mini-batches for optimisation; batch size and per-epoch shuffling of windows are stated in Section 5.3.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [8] with a shared trunk followed by two task-specific branches; the overall layout is shown in Figure 4.2. This structure mirrors the dual-embedding formulation introduced in Section 4.1: the trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings.

Input embedding

Each PDW vector $\boldsymbol{x}_n \in \mathbb{R}^d$, normalised as described in Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\text{in}}\boldsymbol{x}_n + \boldsymbol{b}_{\text{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \tag{4.8}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. No positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$, which makes a separate position signal redundant; consistent with this argument, Gunn et al. [5] report that adding sinusoidal or learnable positional encodings to a transformer deinterleaver yields a small but reproducible drop in performance, attributed to the resulting redundancy with the TOA feature.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 6$ layers are used.

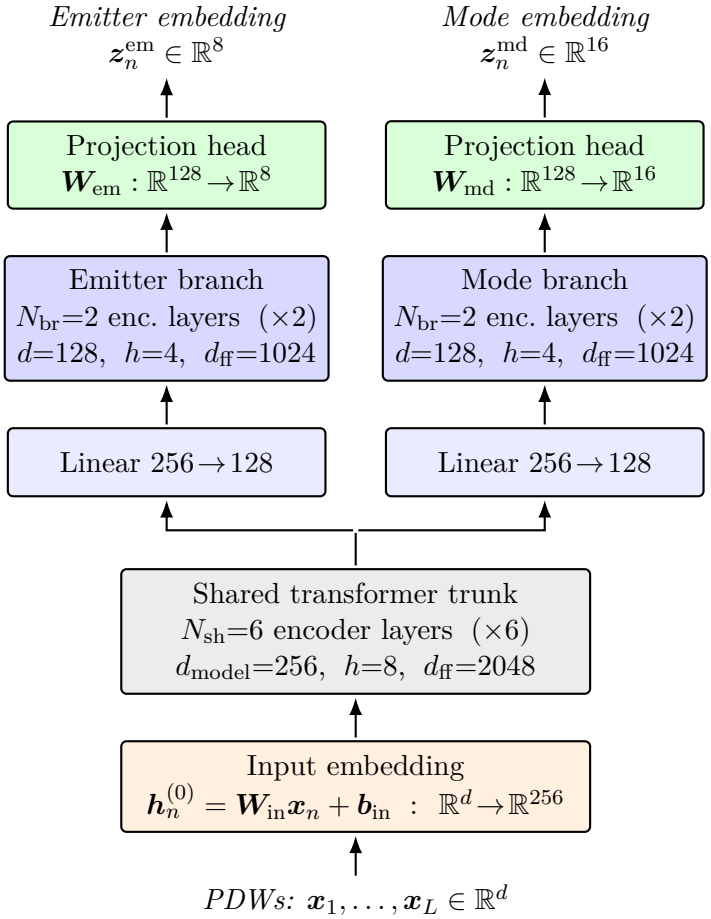

Figure 4.2. Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\mathrm{model}} = 256$ by an input embedding, processed by a shared trunk of 6 transformer-encoder layers, and then split into two parallel branches with their own $256 \to 128$ projection and 2 encoder layers. Branch outputs are mapped by linear projection heads to the emitter embedding space $\mathbb{R}^8$ and the mode embedding space $\mathbb{R}^{16}$.

Its output is a contextualised token sequence $\boldsymbol{H}^{\mathrm{sh}} = (\boldsymbol{h}_1^{\mathrm{sh}}, \ldots, \boldsymbol{h}_L^{\mathrm{sh}})$ with $\boldsymbol{h}_n^{\mathrm{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters, producing the dual embeddings of Equation (4.5). Each branch begins with an affine projection that reduces the hidden width from $d_{\mathrm{model}} = 256$ to $d_{\mathrm{br}} = 128$, after which $N_{\mathrm{br}} = 2$ transformer-encoder layers of the same form as the trunk are applied. Within a branch, MHA uses $h_{\mathrm{br}} = 4$ heads (again with per-head dimension 32) and the FFN inner dimension is $d_{\mathrm{ff,br}} = 1024$. The reduced width and depth reflect a deliberate asymmetry: the trunk is intended to carry the heavy lifting of contextual mixing, while the branches only need to specialise the representation to either emitter identity or operating mode. Write the resulting token sequences as $\boldsymbol{H}^{\mathrm{em}} = (\boldsymbol{h}_1^{\mathrm{em}}, \ldots, \boldsymbol{h}_L^{\mathrm{em}})$ and $\boldsymbol{H}^{\mathrm{md}} = (\boldsymbol{h}_1^{\mathrm{md}}, \ldots, \boldsymbol{h}_L^{\mathrm{md}})$ with $\boldsymbol{h}_n^{\mathrm{em}}, \boldsymbol{h}_n^{\mathrm{md}} \in \mathbb{R}^{128}$.

Projection heads

Each branch terminates in a linear projection head that maps every token to its respective embedding space,

$$\boldsymbol{z}_n^{\mathrm{em}} = \boldsymbol{W}_{\mathrm{em}} \boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}, \qquad \boldsymbol{z}_n^{\mathrm{md}} = \boldsymbol{W}_{\mathrm{md}} \boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}, \tag{4.9}$$

with $\boldsymbol{z}_n^{\mathrm{em}} \in \mathbb{R}^p$ and $\boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^q$ for $p = 8$ and $q = 16$. The mode space is assigned the higher dimensionality because, conditional on emitter identity, mode structure is expected to require a richer geometry to separate subtle waveform regimes, whereas emitter membership induces a coarser partition over the same pulses and is well served by a more compact embedding.

Per-window emitter clustering with HDBSCAN and mode classification

The projection heads in Equation (4.9) map each pulse to a continuous vector, whereas the per-pulse emitter and mode outputs of the system in Section 4.1 are discrete. The two branches differ in how that discretisation is obtained, because of the asymmetric label semantics described in Section 4.1.

For the emitter branch, emitter identifiers are train-local and the inferred count $\hat{K}_{\mathrm{em}}$ in Equation (4.6) is not known a priori on previously unseen pulse trains. HDBSCAN [14, 15] is therefore applied to the emitter embedding trajectory within every analysis window: the L points $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}$ form a self-contained clustering instance, and the effective number of clusters follows the density hierarchy rather than being fixed in advance. As defined in the framework of Campello et al. [14], HDBSCAN replaces the

single global density threshold of DBSCAN [4] with a hierarchy of density estimates: nested level sets are summarised in a condensed tree, and a flat clustering is read off by favouring groups that persist as coherent peaks rather than artefacts of a particular threshold. The same construction supports an explicit noise category for points that are not assigned to any cluster leaf retained after the hierarchy is flattened [14], and the computations use the reference library described by McInnes et al. [15].

For the mode branch, the global catalogue $\{1, \ldots, M\}$ is known and shared across pulse trains, so a fixed classification head $f\colon \mathbb{R}^q \to \mathbb{R}^M$ is attached to $\boldsymbol{z}_n^{\mathrm{md}}$ and the per-pulse prediction $\hat{m}_n$ is the argmax of its logits, as introduced in Equation (4.7). The mode embeddings $\boldsymbol{z}_n^{\mathrm{md}}$ remain available for downstream qualitative analysis (visualisation, nearest-neighbour mode lookup) but the operational mode label is read directly from the classifier rather than from an unsupervised clustering of the mode embeddings.

The collection of all linear maps in Equations (4.8) and (4.9), the classification head f, and the attention and FFN weights of the trunk and branches constitutes the parameter vector $\boldsymbol{\theta}$ that the loss in Section 4.4 optimises.

4.4 Loss Function

The trainable objective uses both label streams introduced in Section 4.1: a within-window *supervised contrastive* loss that uses the train-local emitter labels $\{e_n\}$ to shape $\boldsymbol{Z}^{\mathrm{em}}$, and a per-pulse cross-entropy loss that uses the global mode labels $\{m_n\}$ to shape $\boldsymbol{Z}^{\mathrm{md}}$ through the classification head from Equation (4.7). The two terms are summed with a weight on the mode branch and minimised jointly.

Supervised contrastive viewpoint

Contrastive objectives shape the geometry of an embedding space by encouraging positives (semantically related samples) to lie close together while pushing unrelated negatives apart [6]. In the original SimCLR formulation, positives are constructed without supervision from two stochastic augmentations of the same input, and the loss is closely related to a lower bound on mutual information between two views in contrastive predictive coding [7]. When per-sample labels are available, the same machinery generalises to *supervised contrastive* learning, in which every other sample sharing the anchor's label is treated as a positive and the remaining samples are negatives [16].

The supervised variant is a natural fit for the emitter task in Section 4.1: within each pulse-train window, every pulse carries an emitter label and the positive set for an anchor is exactly the set of other pulses with the same emitter symbol. Because emitter symbols are unique only within a pulse train, contrasts are restricted to the current window, which matches both the train-local semantics of e_n and the windowing strategy in Section 4.2. A large set of negatives drawn from the same window is essential in

either viewpoint: without repulsive pressure on unrelated pairs, encoders can collapse to a constant map and still achieve low training error on a naïve agreement loss [6].

Supervised contrastive building block

Let $\phi(\boldsymbol{z}) = \boldsymbol{z}/\|\boldsymbol{z}\|_2$ denote ℓ_2 normalisation and let $\tau > 0$ denote the contrastive temperature. Write the scaled cosine similarity as

$$s_\tau(\boldsymbol{a}, \boldsymbol{b}) = \frac{1}{\tau}\, \phi(\boldsymbol{a})^\top \phi(\boldsymbol{b}). \tag{4.10}$$

The temperature τ rescales inner products after normalisation: smaller τ sharpens the softmax in Equation (4.11), so the loss concentrates on the hardest negatives and gradients emphasise fine separation on the sphere, whereas larger τ yields a softer distribution closer to uniform and can stabilise optimisation when embeddings are poorly separated early in training [6]. All contrastive terms in this thesis share a fixed $\tau = 0.2$, in line with values commonly used for normalised contrastive features [6, 16].

Given a finite anchor set $\mathcal{I}$ with embeddings $\{\boldsymbol{z}_i\}_{i\in\mathcal{I}}$ and, for each anchor $i \in \mathcal{I}$, a positive set $\mathcal{P}(i) \subseteq \mathcal{I} \setminus \{i\}$ defined by label coincidence, the supervised contrastive contribution averages a per-positive InfoNCE / NT-Xent term over $\mathcal{P}(i)$ [16]:

$$\mathcal{L}_{\mathrm{SC}}\Big(\mathcal{I}, \{\mathcal{P}(i)\}_{i\in\mathcal{I}}\Big) = -\frac{1}{|\mathcal{I}^+|} \sum_{i\in\mathcal{I}^+} \frac{1}{|\mathcal{P}(i)|} \sum_{p\in\mathcal{P}(i)} \log \frac{\exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_p)\Big)}{\displaystyle\sum_{\substack{j\in\mathcal{I}\\ j\neq i}} \exp\Big(s_\tau(\boldsymbol{z}_i, \boldsymbol{z}_j)\Big)}, \tag{4.11}$$

where $\mathcal{I}^+ = \{i \in \mathcal{I} : \mathcal{P}(i) \neq \emptyset\}$ collects anchors with at least one positive available, and the convention $\mathcal{L}_{\mathrm{SC}} = 0$ is used when $\mathcal{I}^+$ is empty. The denominator collects every candidate in $\mathcal{I}$ except the anchor itself, so unrelated pulses act as negatives and collapse is discouraged [6].

Emitter-oriented term

The emitter branch targets a representation in which pulses from the same train-local emitter are close and pulses from different emitters are separated, which is the geometric content expected of a deinterleaving embedding [5]. For a window of L pulses with emitter labels $(e_1, \ldots, e_L)$ and emitter embeddings $\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}$ obtained from Equation (4.9), the within-window positive set for anchor position n is

$$\mathcal{P}_n^{\mathrm{em}} = \{\, n' \in \{1, \ldots, L\} \setminus \{n\} : e_{n'} = e_n \,\}. \tag{4.12}$$

The per-window emitter loss is the supervised contrastive contribution Equation (4.11) on the index set $\{1, \ldots, L\}$ with these positive sets, and the batch loss is the mean over

the B windows in a mini-batch:

$$\mathcal{L}_{\text{em}} = \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}_{\text{SC}}\Big(\{1,\dots,L\},\, \{\mathcal{P}_{b,n}^{\text{em}}\}_{n=1}^{L}\Big), \tag{4.13}$$

where the subscript b in $\mathcal{P}_{b,n}^{\text{em}}$ indicates that positives are determined by the emitter labels of window b. Restricting both anchors and negatives to the current window enforces the train-local semantics of the emitter symbol from Section 4.1: pulses from different pulse trains are never compared, so the loss never attaches meaning to coincidences between emitter integers across windows. A pulse whose emitter is unique inside its window (so $\mathcal{P}_{n}^{\text{em}} = \emptyset$) contributes only as a negative for other anchors and is excluded from the per-window average, following the convention in Equation (4.11).

Mode-oriented term

Operating modes carry meaning across pulse trains, and every pulse in the training data is annotated with a global mode label $m_n \in \{1,\dots,M\}$. A linear classification head $f\colon \mathbb{R}^q \to \mathbb{R}^M$ on top of the mode embedding produces per-class logits, and the mode branch is trained with the standard per-pulse cross-entropy

$$\mathcal{L}_{\text{md}} = -\frac{1}{B\,L}\sum_{b=1}^{B}\sum_{n=1}^{L} \log \frac{\exp\Big(f_{m_{b,n}}\Big(\boldsymbol{z}_{b,n}^{\text{md}}\Big)\Big)}{\displaystyle\sum_{\ell=1}^{M}\exp\Big(f_{\ell}\Big(\boldsymbol{z}_{b,n}^{\text{md}}\Big)\Big)}, \tag{4.14}$$

where $m_{b,n}$ denotes the ground-truth mode label of pulse n in window b and f is the head used at inference in Equation (4.7). Because the catalogue $\{1,\dots,M\}$ is shared across windows, this term applies the same supervision identically to every window, and gradients on f and on $f_{\boldsymbol{\theta}}$ pull together mode embeddings whose ground-truth modes coincide regardless of which physical emitter or which train produced them. Auxiliary contrastive structure on $\boldsymbol{z}_n^{\text{md}}$ (for example, a SupCon term that uses mode labels as positives [16]) is a natural extension when classifier confidence alone is insufficient to organise mode geometry; whether such a term is active is part of the optimisation configuration recorded in Chapter 5.

Combined objective and use of the projection heads

The overall training loss is the weighted sum

$$\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}}\,\mathcal{L}_{\text{md}}, \qquad \lambda_{\text{md}} > 0, \tag{4.15}$$

with λ_{md} treated as a hyperparameter and reported in Chapter 5.

For emitter inference on a previously unseen pulse train, the deliverable is the discrete per-pulse assignment $\hat{e}_n$ in Equation (4.6), obtained by clustering the emitter

embeddings $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ within each test window with the post-hoc clustering algorithm of Section 4.3; once those labels are fixed, storing every $\boldsymbol{z}_n^{\mathrm{em}}$ is optional because the continuous representation is not required for the operational tasks that consume hard emitter tracks. For mode inference, the per-pulse prediction $\hat{m}_n$ is read directly from the classification head as in Equation (4.7), while the mode embeddings $\boldsymbol{z}_n^{\mathrm{md}}$ remain useful for visualisation, nearest-neighbour mode analysis, and any future matching against external waveform libraries.

4.5 Training Procedure

The parameter vector $\boldsymbol{\theta}$ attached to the maps in Equations (4.8) and (4.9) is estimated by minimising the training objective $\mathcal{L}$ in Equation (4.15) on mini-batches of windows. Gradients are obtained by reverse-mode automatic differentiation and used in an adaptive first-order optimiser of the Adam family [17]. Unless Section 5.3 specifies otherwise, the implementation follows the default moment decay rates $(\beta_1, \beta_2) = (0.9, 0.999)$, stabiliser $\epsilon = 10^{-8}$, and bias-corrected running estimates of the first and second moments described by Kingma and Ba [17], as exposed by the PyTorch optimiser interface.

The learning rate follows a cosine decay from a peak value down to a small floor over the course of training, which keeps step sizes comparatively large while the loss landscape is coarse and shrinks them during the final refinement phase. Peak learning rate, weight decay, batch size, number of passes over the training windows, and the exact step counting convention (per epoch versus global step) are hyperparameters and are listed in Table 5.2. At the start of each epoch, training windows are shuffled so that consecutive mini-batches are not ordered by scenario; any fixed random seeds, data-loader worker counts, and reproducibility settings used in the reported runs are stated in Sections 5.1 and 5.3.

Dropout inside the transformer blocks of Section 4.3 follows the usual pattern of stochastic attention and feed-forward masks; dropout rates belong to the same hyperparameter table. Training can be stopped early when a validation score computed on held-out windows (for example the smoothed mini-batch loss or an auxiliary clustering criterion) fails to improve for a fixed patience measured in epochs, in which case the best checkpoint prior to stagnation is retained; patience, validation metric, and checkpoint policy are also recorded in Table 5.2.

If a DEC-style Kullback–Leibler term is added as suggested in Section 4.4, target distributions are recomputed from the student assignments on a schedule analogous to Xie et al. [10], and any sharpening temperature or update period is treated as part of the optimisation protocol reported in Section 5.3. Likewise, the mode-branch weight λ_{md} may be held small during an initial warm-up segment so that the trunk learns coarse temporal structure before the finer mode contrastive gradients dominate; whether such a ramp is used, and its length, is part of the same configuration table. Multi-phase schedules, such as representation pretraining followed by clustering fine-tuning, are

compatible with the same tooling; if they are employed, the phases and their objectives are stated in Section 5.3.

4.6 Evaluation Metrics

Let y_n and $\hat{y}_n$ denote reference and predicted cluster indices for pulse $n \in \{1, \dots, N\}$, as in Equations (4.2), (4.3) and (4.5), instantiated by $(e_n, \hat{e}_n)$ or $(m_n, \hat{m}_n)$. The contingency is $n_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i,\, \hat{y}_n = j]$ with marginals $a_i = \sum_j n_{ij}$ and $b_j = \sum_i n_{ij}$.

Adjusted Rand index

The ARI [18] is

$$\mathrm{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \big[\sum_i \binom{a_i}{2}\big]\big[\sum_j \binom{b_j}{2}\big] \big/ \binom{N}{2}}{\frac{1}{2}\Big[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\Big] - \big[\sum_i \binom{a_i}{2}\big]\big[\sum_j \binom{b_j}{2}\big] \big/ \binom{N}{2}}\,, \qquad \mathrm{ARI} \le 1\,. \tag{4.16}$$

Perfect agreement up to relabelling gives $\mathrm{ARI} = 1$.

Entropies, mutual information, and normalizations

Shannon entropies of the marginals are

$$H(Y) = -\sum_{i:\, a_i > 0} \frac{a_i}{N} \log \frac{a_i}{N}\,, \qquad H(\hat{Y}) = -\sum_{j:\, b_j > 0} \frac{b_j}{N} \log \frac{b_j}{N}\,. \tag{4.17}$$

The empirical mutual information is [19]

$$\mathrm{MI}(Y; \hat{Y}) = \sum_{i,j:\, n_{ij} > 0} \frac{n_{ij}}{N} \log \frac{N\, n_{ij}}{a_i b_j}\,, \tag{4.18}$$

with $0 \log 0 = 0$. A NMI score is obtained by dividing MI by a function of $\{H(Y), H(\hat{Y})\}$; we use the arithmetic-mean denominator [19],

$$\mathrm{NMI}(Y; \hat{Y}) = \frac{2\,\mathrm{MI}(Y; \hat{Y})}{H(Y) + H(\hat{Y})}\,, \qquad H(Y) + H(\hat{Y}) > 0\,, \tag{4.19}$$

and set $\mathrm{NMI} = 0$ if $H(Y) + H(\hat{Y}) = 0$. Let $\mathbb{E}[\mathrm{MI}]$ be the expectation of MI under the permutation null that fixes one partition while permuting labels subject to fixed cluster sizes [19]. The AMI is

$$\mathrm{AMI}(Y; \hat{Y}) = \frac{\mathrm{MI}(Y; \hat{Y}) - \mathbb{E}[\mathrm{MI}]}{\frac{1}{2}\big(H(Y) + H(\hat{Y})\big) - \mathbb{E}[\mathrm{MI}]}\,, \tag{4.20}$$

when the denominator is positive; otherwise AMI $\equiv 0$.

Clustering accuracy

Cluster indices are arbitrary up to a bijection ϕ on label symbols. The ACC is the largest agreement after an optimal relabeling,

$$\mathrm{ACC} = \frac{1}{N} \max_{\phi \in \Phi} \sum_{n=1}^{N} \mathbf{1}\big[y_n = \phi(\hat{y}_n)\big] , \tag{4.21}$$

where Φ is the set of bijections between the active reference labels and the active predicted labels when their counts coincide; if cardinalities differ, the same maximum is computed by the Hungarian (linear-sum assignment) algorithm on the matrix (n_{ij}) with cost $-n_{ij}$ [20].

V-measure

With conditional entropies

$$H(Y \mid \hat{Y}) = - \sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j} , \qquad H(\hat{Y} \mid Y) = - \sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i} , \tag{4.22}$$

homogeneity and completeness are [21]

$$h = 1 - \frac{H(Y \mid \hat{Y})}{H(Y)} , \qquad c = 1 - \frac{H(\hat{Y} \mid Y)}{H(\hat{Y})} , \tag{4.23}$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is

$$\mathrm{V}(Y, \hat{Y}) = \frac{2hc}{h + c} , \qquad h + c > 0 , \tag{4.24}$$

and $\mathrm{V} = 0$ if $h = c = 0$.

Figures

Two-dimensional projections of $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}$ coloured by (e_n, m_n) complement Equations (4.16), (4.19) to (4.21) and (4.24); protocols and numbers appear in Chapter 5.

4.7 Baseline Methods

The baselines isolate the contribution of the dual-branch layout in Section 4.3 and of the mode-oriented term $\mathcal{L}_{\mathrm{md}}$ in Equation (4.15) relative to a single-stack encoder and to classical clustering without a sequence model.

The first neural baseline removes the emitter and mode branches and replaces them with one stack of eight transformer-encoder layers of the same functional form as the shared trunk (hidden width, head count, and FFN expansion match that trunk so the comparison stresses topology and objective rather than a change in per-layer width). Token embeddings $\boldsymbol{h}_n^{(0)}$ from Equation (4.8) pass through this stack; a single linear head then maps each contextualised token to $\boldsymbol{z}_n \in \mathbb{R}^p$, using the same output dimension p as the emitter branch in the full model. Training minimises only the emitter-oriented contrastive loss $\mathcal{L}_{\text{em}}$ in Equation (4.13); $\mathcal{L}_{\text{md}}$ and its inter-window structure are omitted, so no auxiliary geometry is trained for operating modes. This configuration tests whether one representation, optimised purely for window-level deinterleaving, suffices for both emitter- and mode-level clusterings reported in Chapter 5.

The second baseline is HDBSCAN [15], a hierarchical density-based method that does not estimate a transformer. It clusters fixed-length vectors built deterministically from the per-feature scaled PDW windows in Section 4.2, for example by per-channel summary statistics or a flattened layout of pulses and features; the exact vectorisation is fixed in Chapter 5. HDBSCAN separates gains due to learned sequence context from what density-based Euclidean clustering already recovers on hand-specified summaries.

Further comparisons—k-means on raw window vectors or on autoencoder embeddings, DEC-style objectives when centroids are used, SwAV-style assignment prediction [11] as an unsupervised-clustering reference, and a fully supervised classifier upper bound that uses the global mode catalogue directly—are listed with their configurations in Chapter 5.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory; some system utilities report a capacity closer to 82 GB because of binary versus decimal gigabyte conventions). The implementation uses Python with PyTorch; exact package versions are pinned in the environment file shipped with the thesis code repository when that repository is published.

Random seeds control weight initialization, training-window shuffling each epoch, and stochastic augmentations. Unless an experiment explicitly varies the seed, the same seed set is reused so that ablations in Section 5.5 are comparable under identical noise conditions. Concrete seed integers, per-run overrides, and mean $\pm$ standard deviation over multiple seeds for quantitative scores are reported with the tables in Section 5.4.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth labels for emitter identity (train-local, in the sense of Section 4.1) and operating mode (drawn from the global catalogue); both label streams are consumed by the training objective in Section 4.4 and are also used to compute the evaluation metrics in Sections 4.6 and 5.4. When a run includes receiver-side degradations (dropped or spurious pulses), they are applied to the ordered pulse lists as in Section 4.2.2 and Algorithm 1 before windowing, and the tables below state which regimes are active. Operational ELINT recordings are not included, which sidesteps distribution shift and disclosure constraints while still exercising mixture traffic, mode changes within emitters, and variable train density.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; all normalisation statistics in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses. Training windows use stride $L/2$ with $L = 1024$ pulses, whereas validation and test windows use stride L, so the effective number of windows per scenario differs by split even when raw pulse counts are similar.

Table 5.1. Synthetic scenario corpus used for training and evaluation. Dashes mark quantities still being reconciled with the export manifest.

Split	Scenarios	Pulses (approx.)	Distinct emitters	Distinct modes
Train	—	—	—	—
Validation	—	—	—	—
Test	—	—	—	—
All	—	—	—	—

Table 5.1 summarises corpus-level figures. The mode column counts distinct global mode symbols that appear anywhere in the corresponding split and is an upper bound on the number of catalogue entries that the mode classifier in Equation (4.7) ever has to discriminate. The emitter column counts the union of train-local emitter symbols, summed across scenarios in the split: because emitter identifiers are unique only within a pulse train (Section 4.1), this number describes total emitter incidence rather than a global set, and the within-window clustering at inference operates on the train-local sub-counts. Scenario design and traffic rules also skew how often each emitter and each mode is exercised, so aggregate pulse counts need not match intuitive "balanced" priors even when the simulator exposes many labels. Because the encoder sees only windows of fixed length L (Section 4.2), rare combinations of emitter, mode, and interleaving pattern can be missing from individual windows or represented by only a handful of pulses, which limits what any single forward pass can evidence about those labels. The numeric entries are placeholders until the final export from the simulator pipeline is frozen; the intended reporting convention is scenarios for volume, pulses for aggregate exposure, and distinct labels for emitter and mode coverage.

5.2.1 Qualitative properties of the synthetic corpus

Beyond aggregate counts, the corpus is checked qualitatively before large training runs: example timelines are inspected for plausible PRI structure per mode, for physically admissible RF and pulse-width ranges, and for the intended mixture density when multiple emitters are active. When receiver-side degradations are enabled (Section 4.2.2), spot checks confirm that dropped pulses thin pulse trains without reordering surviving pulses and that spurious insertions remain within the configured rule set. This mirrors the reporting practice in earlier Saab-hosted simulator studies, which often include illustrative emitter tracks alongside tables [12]. Section 5.6 complements the brief sanity checks here with embedding-space visualizations and other qualitative diagnostics once model checkpoints are available.

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	Adam [17] with PyTorch defaults $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	—
Minimum learning rate $\eta_{\min}$	0 (cosine floor; PyTorch default)
Cosine period $T_{\max}$	30 scheduler steps, one step per training epoch
Maximum epochs	30
Early stopping patience	7 epochs without improvement on the validation loss
Early stopping checkpoint	Weights from the epoch with lowest validation loss so far
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	—
Mode loss weight λ_{md}	—
Weight decay	—

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3 to 4.5. Unless Section 5.5 states otherwise, baseline models use the same epoch budget, hardware (Section 5.1), and early-stopping protocol so that differences reflect architecture and objective rather than compute.

The contrastive temperature is fixed at $\tau = 0.2$ as in Section 4.4. Table 5.2 summarises the optimisation schedule and regularisation for the proposed dual-branch encoder; entries marked with an em dash are still being matched to the released training script and will be filled before the final thesis submission.

The cosine scheduler follows the PyTorch `CosineAnnealingLR` convention: after each epoch the learning rate is updated from $\eta_{\max}$ toward $\eta_{\min}$ over $T_{\max} = 30$ steps, which aligns the period with the nominal training horizon. If early stopping triggers first, training halts at the best validation checkpoint and the final few scheduler steps may not be executed.

5.4 Quantitative Results

This section reports primary clustering scores for the proposed model and for each baseline under the data conditions in Section 5.2. Rows index methods (including ablated variants where applicable); columns index emitter-level and mode-level NMI, ARI, and ACC after optimal alignment of cluster indices to reference labels where those labels exist only for evaluation (Section 4.6). Each entry is aggregated over

random seeds and window splits as stated in Sections 5.1 and 5.3; mean ± one standard deviation is shown when multiple seeds are available.

5.5 Ablation Studies

Ablations isolate the contribution of individual components in Chapter 4 by retraining with exactly one change relative to the full configuration (matched data, seeds, and hyperparameters unless the ablation itself is a hyperparameter). Planned comparisons include removing or downweighting the emitter contrastive term, the mode contrastive term, or the coupling between branches; reducing the number of attention layers or heads; disabling stochastic augmentations beyond deterministic scaling; and swapping HDBSCAN post-processing for a simpler fixed-k assignment when that variant is implemented.

5.6 Qualitative Analysis

Qualitative analysis complements the scalar metrics in Section 5.4 by checking whether embeddings organize pulses in ways that align with physical emitter boundaries and with mode transitions inside a window. The checks in Section 5.2.1 validate that the simulator export behaves as intended; this section applies analogous scrutiny *after* models are trained.

Two-dimensional projections of pooled or per-pulse embeddings (for example UMAP or t-SNE) should be colored separately by reference emitter and by reference mode when labels exist only for evaluation. Disjoint emitter-colored manifolds with mode-colored substructure that stays coherent within an emitter are evidence that the dual geometry in Chapter 4 is behaving as designed; elongated bridges between emitter manifolds often indicate residual deinterleaving ambiguity or shared waveform parameters across platforms.

Attention maps or head-wise statistics, if exported, can be overlaid on TOA-ordered pulses to see whether specific heads track stable intervals versus rapid mode hops.

Confusion matrices between inferred clusters and reference labels (after optimal assignment) are optional but useful when a small number of emitter or mode classes dominate error mass.

CHAPTER 6

Discussion

6.1 Interpretation of Results

This section connects the quantitative tables in Section 5.4 and the qualitative material in Section 5.6 to the modeling choices in Chapter 4. The goal is not to restate individual scores, but to explain *which* mechanisms plausibly drive the observed ranking of methods and ablations.

When emitter-level NMI or ARI lag while mode-level scores are stronger, a natural hypothesis is that the encoder prioritizes intra-window structure tied to waveform parameters (which modes share) before it fully exploits slower cues that separate physical platforms across mixed traffic. Conversely, if emitter scores dominate but mode partitions collapse, the dual-head coupling or the mode contrastive term may be underweighted relative to the emitter branch, or certain modes may be too short-lived inside length-L windows to support stable clusters.

Noise and spurious-pulse regimes from Section 4.2.2 interact with self-attention in two ways: they perturb local timing patterns that define modes, and they change which pulses co-occur inside a window, which can confuse any window-level emitter summary. Interpretation should therefore read noise sweeps alongside the corpus diagnostics in Sections 5.2.1 and 5.6: a metric drop that coincides with visibly corrupted tracks is evidence of a robustness limit, whereas a drop on clean held-out scenarios points toward generalization or optimization issues instead.

6.2 Comparison with Related Work

Relative to classical ELINT fingerprinter pipelines surveyed in Section 3.1, the proposed approach trades hand-engineered feature logic for a learned encoder, at the cost of data hunger, compute, and reduced transparency unless accompanied by diagnostics such as those in Section 5.6. Unlike the supervised radar classifiers discussed in Section 3.2, which typically assume a fixed global label set for emitters, the training objective here treats emitter identity as a *train-local* symbol used only through within-train comparisons (Section 4.4); operating modes, by contrast, are predicted against the same kind of global catalogue used in those classifiers.

Deep clustering methods in Section 3.3 typically optimise a *single* flat partition without labels, or attach two unrelated supervised heads to the same backbone. This thesis instead targets two aligned partitions with a nesting interpretation (modes within emitters), with asymmetric supervision: a within-train supervised contrastive signal

for the emitter branch and a global classification signal for the mode branch. The setup is closer in spirit to offline pulse-level grouping work that reasons about geometry and overlap in ESM data [22] but uses gradient-based representation learning rather than hand-tuned density stages alone. Where scores fall short of the best published supervised benchmarks, the gap is expected: those models often operate on curated single-emitter tracks with a flat, globally consistent emitter label set, whereas the present setting stresses mixture windows and a label structure that forces emitter inference through clustering rather than through a global classifier.

6.3 Limitations

The experimental programme is offline: models are trained and scored on bounded, revisit-able windows rather than under the latency, memory, and non-stationarity constraints of a continuously arriving pulse stream (Sections 1.4, 1.4.2 and 4.2). That setting supports controlled optimization and reproducible metrics but does not demonstrate that the same representations would support stable online emitter or mode tracking without periodic retraining or revised windowing policy.

Fixed-length windows are partly a consequence of full self-attention, whose standard implementation scales quadratically with the number of pulses in the window (Section 4.2). The chosen cap on L therefore trades faithful coverage of long mode segments and rare emitters against compute, and the present pipeline drops trailing segments shorter than L, which can remove evidence entirely when a scenario does not fill an integral number of windows.

Interleaved emitters and mode-hopping further skew how many pulses from each ground-truth label appear inside any given window, so performance summaries aggregate over heterogeneous label incidence rather than over an idealized balanced design (Section 5.2). Where metrics depend on comparing inferred partitions to reference labels, conclusions are most informative for label configurations that actually recur with sufficient mass inside windows; weak support for a label in a window is a limitation of the observation model, not necessarily of the encoder in isolation.

Finally, all quantitative evidence is derived from synthetic scenarios with full reference labels; operational recordings with imperfect deinterleaving, sensor dropouts, and distribution shift are not included here. Related Saab-line work on track association under deinterleaving errors highlights that real pipelines fragment pulse trains and introduce structured mistakes that are only partly captured by the stylized drop and spurious models used for stress tests [23]. Likewise, classifiers trained on simulator draws can exhibit overconfidence on held-out emitters or modulation families, which motivates explicit out-of-distribution and drift analyses when moving toward fielded receivers [24]. The present thesis does not implement drift monitors or uncertainty heads; those remain orthogonal safeguards if embeddings from Chapter 4 are ever deployed under evolving threat libraries.

The HDBSCAN post-processing used in Section 4.3 also encodes density assumptions that may not match every deployment, and we do not exhaust efficient-attention or hierarchical alternatives that could extend context within the same hardware envelope.

6.4 Implications

For ELINT and ESM practice, the main takeaway is methodological: joint emitter- and mode-aware embeddings can be learned from PDW streams whose supervision is structured asymmetrically (train-local emitter labels, global mode catalogue), but their reliability still hinges on scenario coverage, windowing policy, and honest reporting of failure modes under mixture and noise (Section 6.3). Operators should treat the resulting per-pulse emitter clusters as hypotheses that require human or library-based adjudication before tactical use, especially when training data are synthetic and drift relative to live emitters is unmodelled; the same caution applies to mode classifications whose confidence is not separately calibrated.

For the deep-clustering and sequence-modeling research communities, the thesis sits at the intersection of contrastive representation learning on irregular pulse sequences and multi-partition clustering with a physical nesting constraint (modes within emitters). The architectural and loss choices in Chapter 4 are portable to other domains with hierarchical latent structure, provided that evaluation metrics separate which level of the hierarchy is being scored.

Follow-up work naturally includes streaming or sliding-window evaluation, richer deinterleaving and track-error models [23], and explicit ML safeguards against dataset shift and overconfident extrapolation [24]. On the ethics side, the considerations in Section 1.6 still apply: dual-use sensitivity, data stewardship if operational traces are introduced later, and the energy cost of transformer-scale training should be revisited whenever the experimental footprint grows.

CHAPTER 7

Conclusion

7.1 Summary

Placeholder paragraph.

7.2 Answers to the Research Questions

Placeholder paragraph.

7.3 Future Work

Placeholder paragraph.

References

[1] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.

[2] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[3] Z. Qu, J. Zhang, Y. Zhou, and L. Ni, "The intelligent evolution of radar signal deinterleaving: A systematic review from foundational algorithms to cognitive AI frontiers", *Sensors*, vol. 26, no. 1, 2026. DOI: 10.3390/s26010248

[4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[5] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: 2503.13476 [cs.LG].

[6] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[7] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: 1807.03748 [cs.LG].

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding", in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[10] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[11] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] T. Jonsson, "Classification of Radar Emitters using Semi-Supervised Contrastive Learning", Host company Saab AB; supervised learning experiments on pulse data from the OpenModesty scenario simulator, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2023.

[13] A. Svensson, “Classification of Radar Emitters Based on Pulse Repetition Interval using Machine Learning”, Host company Saab AB; synthetic PRI sequences from specification-driven generators with controlled noise regimes, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.

[14] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, “Hierarchical density estimates for data clustering, visualization, and outlier detection”, *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: 10.1145/2733381

[15] L. McInnes, J. Healy, and S. Astels, “hdbscan: Hierarchical density based clustering”, *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: 10.21105/joss.00205

[16] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning”, in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[17] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, arXiv preprint arXiv:1412.6980, 2017. arXiv: 1412.6980 [cs.LG].

[18] L. Hubert and P. Arabie, “Comparing partitions”, *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[19] N. X. Vinh, J. Epps, and J. Bailey, “Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance”, *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[20] H. W. Kuhn, “The Hungarian method for the assignment problem”, *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955. DOI: 10.1002/nav.3800020109

[21] A. Rosenberg and J. Hirschberg, “V-measure: A conditional entropy-based external cluster evaluation measure”, in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

[22] S. Bedoire, “Offline direction clustering of overlapping radar pulses from homogeneous emitters”, Host company Saab; DBSCAN-style clustering on pulse directions with simulated scenarios, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.

[23] V. Jakobsson, “Offline radar pulse track association with deinterleaving errors”, Host company SAAB; time-series clustering for track fragments under imperfect deinterleaving, Master’s thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

[24] K. Coleman, “Dataset drift in radar warning receivers: Out-of-distribution detection for radar emitter classification using an RNN-based deep ensemble”, UPTEC F 23041, 30 hp; simulator data from Saab AB and drift/OOD analysis for emitter classification, Master’s thesis, Uppsala University, Uppsala, Sweden, 2023.

APPENDIX A

Additional Results

Placeholder paragraph.

APPENDIX B

Implementation Details

Placeholder paragraph.

APPENDIX C

Notation Reference

Placeholder paragraph.